

Multi-Agent RAG Document Intelligence System

Complete 4-Week Build Roadmap · Job-Ready AI Engineering · Asmad Nadeem · Summer 2026

WHAT YOU ARE BUILDING

THE PROJECT	THE STACK	THE RESULT
An AI system that reads any PDF you upload, understands it, and answers your questions by pulling relevant information from the document.	Python · pdfplumber · LangChain · Google Gemini API · ChromaDB · FastAPI · PostgreSQL · Streamlit · Docker · Railway (deployment)	A web application that lets you upload a PDF, ask questions, and get answers in a chat interface.

TOOL GLOSSARY — READ THIS FIRST

Every tool explained in plain English before you ever use it.

TOOL	WHAT IT ACTUALLY IS
Python	The programming language you write everything in. Like giving instructions to a computer in a structured way. You already have this — check by typing <code>python --version</code> in terminal.
VS Code	The app where you write your code. Like Microsoft Word, but for code. Download free from code.visualstudio.com . You will spend 90% of your time here.
Terminal	A black text window where you type commands instead of clicking. Open it inside VS Code via Terminal → New Terminal. This is how you run your code and install tools.
pip	Python's tool for installing libraries. When you type <code>pip install something</code> , it downloads and installs that tool for you. Comes with Python automatically.
pdfplumber	A Python library that opens PDF files and pulls the text out. It reads each page and gives you the text as a string you can work with.
Chunking	Splitting a long document into smaller pieces (e.g. 500 words each). You cannot feed an entire PDF to AI at once — it is too large. Chunks are what get searched and retrieved.
Embedding	Turning text into a list of numbers that represents its meaning. Two sentences about similar topics get similar number lists. This is how AI finds relevant chunks without matching exact words.
Gemini API	Google's AI service. You send it text, it sends back intelligent responses. You already have the API key. This is the brain of your system.
ChromaDB	A database built specifically to store embeddings (the number lists). When you search, it finds which stored numbers are most similar to your question's numbers. Runs locally, completely free.
LangChain	A Python library that connects everything together — the AI model, the vector database, the document loader. It handles the plumbing so you focus on logic.
FastAPI	A Python library for building a web backend. It turns your Python script into something with a real web address that other apps can talk to. Like giving your script a front door.
PostgreSQL	A real database for storing structured data permanently — like which documents a user uploaded, when, and metadata. Think of it as a very powerful spreadsheet your code can read and write to.
Streamlit	A Python library that turns your backend into a simple website with almost zero HTML or CSS knowledge needed. This is your frontend — what users actually see and click.
Docker	A way to package your app and everything it needs into a container that runs identically anywhere — your laptop, a server, anyone else's machine. Solves the 'it works on my machine' problem.
Railway	A hosting platform with a free tier. You connect your GitHub repo, it builds and deploys your Docker container, and gives you a live public URL. No server management needed.

.env file	A file where you store secret keys like your Gemini API key. Never upload this to GitHub. python-dotenv is the library that reads it into your code safely.
RAG	Retrieval-Augmented Generation. The technique your whole project is built on. Instead of AI answering from its own training, it retrieves relevant chunks from YOUR document and answers from those. This means accurate, cited answers.
Agent	An AI that can use tools, make decisions, and take actions. Instead of one AI call, multiple specialized agents each do one job — retrieve, verify, cite — and pass results between them.

NON-NEGOTIABLE RULES

DO THIS	NEVER DO THIS
Struggle with a problem for 30 min before asking AI	Paste code you do not understand and run it blindly
Read every line of code before running it — look up what you don't understand	Copy-paste solutions from Stack Overflow or AI without reading them
After each day, close everything and explain out loud what you built	Skip the explanation test — if you cannot explain it, you do not know it
Keep a one-line build log per day: what you built, what broke, how you fixed it	Do not fix it the next day if something is broken — fix it first
Ask AI: 'explain why X works' or 'what is wrong with my code, do not fix it'	Ask AI to 'write me X' and paste the result
Test your own code trying to break it — bad inputs, empty files, wrong file names	Only test the happy path (the thing that is supposed to work)
Commit to GitHub at the end of every single day	Wait until the end of the week to push code
Add a .gitignore file that excludes .env — never commit API keys	Upload your .env file to GitHub even once

ONE-TIME SETUP — DO BEFORE DAY 1

1	Install Python	Go to python.org → Download → Install. CRITICAL: tick 'Add Python to PATH' during install on Windows. After install, open terminal and type: python --version You should see something like Python 3.11.x. If you do not, reinstall.
2	Install VS Code	Go to code.visualstudio.com → Download → Install. Open it. Go to Extensions (left sidebar) → search 'Python' → install the Microsoft Python extension.
3	Create project folder	Create a folder called rag-project somewhere on your computer (Desktop is fine). Open VS Code → File → Open Folder → select rag-project. Now open Terminal inside VS Code: Terminal → New Terminal. You should see a command prompt inside VS Code.
4	Set up virtual environment	In the VS Code terminal, type: python -m venv venv This creates an isolated Python environment so your project's libraries don't conflict with anything else. Activate it: Windows: venv\Scripts\activate You should see (venv) appear at the start of your terminal line.
5	Get your Gemini API key ready	You already have this from the n8n project. Go to aistudio.google.com → API Keys → copy your key. Create a file named exactly .env in your rag-project folder and add this line: GEMINI_API_KEY=your_actual_key_here
6	Create .gitignore	Create a file named .gitignore in rag-project. Add these lines: .env venv/ __pycache__/* *.pyc This tells Git to never upload these files. The .env file especially must never go to GitHub.
7	Initialize GitHub repo	Go to github.com → New Repository → name it rag-document-qa → initialize with README. Then in VS Code terminal: git init git remote add origin YOUR_REPO_URL Commit and push at the end of every day.

No web server, no deployment. Just Python scripts. You are building the brain of the system first.

WHAT YOU WILL HAVE BY END OF WEEK 1

A Python script that reads any PDF → splits it into chunks → turns chunks into embeddings → stores them in ChromaDB → you ask a question → it finds the right chunks → Gemini answers from those chunks with citations. This IS a RAG pipeline.

DAY 1	GOAL: Read a PDF and print its text	TOOLS: pdfplumber
	1. In terminal (with venv active): pip install pdfplumber 2. Drop any PDF into rag-project folder, rename it test.pdf 3. Create main.py, write this code: import pdfplumber with pdfplumber.open('test.pdf') as pdf: for page in pdf.pages: print(page.extract_text()) 4. Run: python main.py 5. You should see the PDF text printed	BUILD: PDF text printed in terminal. If you see garbage characters, try a different PDF.
		WATCH: Error 'file not found' means your PDF is not in the right folder. Error 'module not found' means pip install did not work — try again.
DAY 2	GOAL: Split the text into chunks	TOOLS: Pure Python
	Add this function to main.py BELOW your existing code: def chunk_text(text, chunk_size=500, overlap=50): chunks = [] start = 0 while start < len(text): end = start + chunk_size chunks.append(text[start:end]) start += chunk_size - overlap return chunks Call it on your extracted text. Print: how many chunks, and the first 2 chunks. Read the chunks — do they cut mid-sentence? That is EXPECTED and normal.	BUILD: Chunks printed. You know what chunk_size and overlap mean.
		WATCH: Do not move on until you can explain: why do we need chunks? Why overlap?
DAY 3	GOAL: Combine all pages + test on a messy PDF	TOOLS: pdfplumber
	Right now you only print per page. Fix main.py so it: 1. Extracts text from ALL pages into one big string 2. Passes that full string to chunk_text() 3. Print total chunks count Then: find a PDF with tables, multiple columns, or lots of formatting (research paper). Run your script on it. Observe: does the text come out clean or jumbled?	BUILD: Full document chunked. Messy PDF limitation observed and noted in build log.
		WATCH: Note the limitation — you do not need to fix it yet. Understanding limits is part of engineering.

DAY 4	GOAL: Get embeddings from Gemini API 1. pip install google-generativeai python-dotenv 2. Make sure your .env file has GEMINI_API_KEY=your_key 3. Add this to main.py: import os from dotenv import load_dotenv import google.generativeai as genai load_dotenv() genai.configure(api_key=os.getenv('GEMINI_API_KEY')) result = genai.embed_content(model='models/embedding-001', content='This is a test sentence.') print(result['embedding'][:5]) 4. Run — you should see 5 decimal numbers printed	TOOLS: google-generativeai, python-dotenv BUILD: List of numbers printed — that is one embedding. Understand: those numbers = the meaning of that sentence. WATCH: If API key error: check .env file has no spaces around the = sign. Check load_dotenv() is called before genai.configure().
DAY 5	GOAL: Embed all chunks Loop over your chunks from Day 2-3 and generate an embedding for each. Store them as a list of dicts: embedded_chunks = [] for chunk in chunks: embedding = genai.embed_content(model='models/embedding-001', content=chunk)['embedding'] embedded_chunks.append({'text': chunk, 'embedding': embedding}) print(f'Embedded {len(embedded_chunks)} chunks') Note: this makes one API call per chunk. If you have 100 chunks, that is 100 calls. Free tier handles this fine.	TOOLS: google-generativeai BUILD: All chunks embedded. Count printed. You can explain what each dict contains. WATCH: Rate limit error: add time.sleep(0.5) inside the loop to slow down requests.
DAY 6	GOAL: Store in ChromaDB + search 1. pip install chromadb 2. Add to main.py: import chromadb client = chromadb.Client() collection = client.create_collection('my_docs') for i, item in enumerate(embedded_chunks): collection.add(documents=[item['text']], embeddings=[item['embedding']], ids=[f'chunk_{i}']) 3. Now search: question = 'your question about the PDF' q_embed = genai.embed_content(model='models/embedding-001', content=question)['embedding'] results = collection.query(query_embeddings=[q_embed], n_results=3) print(results['documents']) 4. Do the top 3 chunks actually contain the answer to your question?	TOOLS: chromadb BUILD: Chunks stored in ChromaDB. Search working. Relevant chunks returned for your question. WATCH: If wrong chunks returned: try rephrasing the question. Investigate — is it a chunking problem or retrieval problem?

DAY 7	GOAL: Wire it all together — full RAG pipeline	TOOLS: google-generativeai, chromadb
	Combine everything into one clean flow: 1. Load PDF → extract text → chunk it → embed chunks → store in Chroma 2. Ask a question → embed question → retrieve top 3 chunks → build context string 3. Send context + question to Gemini: context = <code>\n\n'.join(results['documents']][0])</code> model = <code>genai.GenerativeModel('gemini-1.5-flash')</code> prompt = <code>f'Answer using ONLY this context. Say which part you used.\n\nContext:\n{context}\n\nQuestion: {question}'</code> response = <code>model.generate_content(prompt)</code> print(response.text) 4. Test with 3 different questions about your PDF	BUILD: End-to-end RAG working. PDF → question → cited answer. This is the foundation of everything. WATCH: Watch for: AI making up answers not in the PDF — this means retrieval is failing. Debug the chunks first.

WEEK 1 CHECKPOINT — Answer these out loud before moving to Week 2

- Why chunk_size=500 and not 50 or 5000? What breaks at extremes?
- What is overlap and what happens if it is 0?
- What does an embedding vector actually represent in plain English?
- When wrong chunks come back — is that a chunking, embedding, or retrieval problem?
- What does your system do if the answer is not in the PDF at all?

WEEK 1 LINKEDIN POST

Built a RAG pipeline from scratch this week — no shortcuts, no LangChain yet. Wrote my own chunking logic, generated embeddings via Gemini API, and got semantic retrieval working with ChromaDB. Source-cited answers from any PDF. The debugging when wrong chunks came back was more valuable than the build itself.

WEEK 2 FastAPI Backend + PostgreSQL — Give It a Real Front Door

Right now your system only runs when you manually run a script. This week you give it a proper web backend — a real API other apps can talk to.

WHAT YOU WILL HAVE BY END OF WEEK 2

A running web server with two endpoints: /upload (accepts a PDF file) and /query (accepts a question, returns an AI answer). Postgres storing document metadata. Testable via browser and Postman.

DAY 1-2	GOAL: Build your first FastAPI endpoints	TOOLS: fastapi, uvicorn
	1. pip install fastapi uvicorn python-multipart 2. Create a new file api.py: from fastapi import FastAPI, UploadFile app = FastAPI() @app.post('/upload') async def upload_pdf(file: UploadFile): contents = await file.read() return {'filename': file.filename, 'status': 'received', 'size': len(contents)} @app.post('/query') async def ask_question(question: str): return {'answer': 'placeholder — will connect RAG here'} 3. Run: uvicorn api:app --reload 4. Open browser: http://127.0.0.1:8000/docs 5. Test both endpoints from that page	BUILD: FastAPI server running. Both endpoints responding. /docs page working.
DAY 3		WATCH: uvicorn is what actually runs your app — FastAPI just defines what happens. --reload means server restarts automatically when you save.
	GOAL: Design your Postgres database tables	TOOLS: Paper first, then PostgreSQL
	SKETCH ON PAPER FIRST — seriously. Design two tables: documents table: id, filename, upload_date, file_size, status chunks table: id, document_id, chunk_text, chunk_index Only after sketching, set up Postgres: Option A (recommended for beginners): Use Supabase — go to supabase.com, create free account, create new project. You get a hosted Postgres with a connection URL. No installation needed. Option B: Install Postgres locally from postgresql.org Save your connection URL — you will need it in Day 4.	BUILD: Tables sketched. Postgres connection URL saved.
		WATCH: Why two tables? Because one document has many chunks. If you merge them, you duplicate document info for every chunk. Bad design.
DAY 4	GOAL: Connect FastAPI to PostgreSQL	TOOLS: psycopg2, SQLAlchemy
	1. pip install psycopg2-binary sqlalchemy 2. Add database connection to api.py: from sqlalchemy import create_engine, text DATABASE_URL = os.getenv('DATABASE_URL') # add this to .env engine = create_engine(DATABASE_URL) 3. Create the tables: with engine.connect() as conn: conn.execute(text("CREATE TABLE IF NOT EXISTS documents (id SERIAL PRIMARY KEY, filename TEXT, upload_date TIMESTAMP DEFAULT NOW(), status TEXT) "")) conn.commit() 4. In your /upload endpoint, insert a row into documents table after receiving file	BUILD: File uploaded → row appears in documents table in Postgres. Verify by checking Supabase dashboard or running SELECT * FROM documents;
		WATCH: Never hardcode your DATABASE_URL in the code. Always use .env and os.getenv().

DAY 5	GOAL: Connect RAG pipeline to FastAPI	TOOLS: All Week 1 code + FastAPI
	Now wire your Week 1 RAG code into the API: 1. When /upload receives a PDF → extract text → chunk it → embed chunks → store in Chroma → log to Postgres 2. When /query receives a question → embed it → retrieve from Chroma → send to Gemini → return answer Move your Week 1 functions into a separate file called rag_pipeline.py Import and call them from api.py This is called separation of concerns — each file does one job	BUILD: Upload a PDF via /docs page → ask a question via /query → get a real AI answer back. End to end through the API.
DAY 6	GOAL: Error handling	TOOLS: FastAPI HTTPException
	Deliberately break your API and handle the errors cleanly: 1. Upload a .txt file instead of PDF → should return clear error 2. Upload empty file → should return clear error 3. Ask empty question → should return clear error 4. Add these checks: from fastapi import HTTPException if not file.filename.endswith('.pdf'): raise HTTPException(status_code=400, detail='Only PDF files supported') if len(contents) == 0: raise HTTPException(status_code=400, detail='File is empty') For every endpoint, think: what can go wrong? Handle it.	BUILD: All error cases return clean messages, not server crashes.
DAY 7	GOAL: Test with Postman + fix what breaks	TOOLS: Postman (free download)
	1. Download Postman from postman.com (free) 2. Send POST requests to /upload with a real PDF file 3. Send POST requests to /query with different questions 4. Try every weird input: huge PDF, PDF with only images, special characters in question 5. Fix every crash and edge case you find 6. Document what you found in your build log	BUILD: API stable under weird inputs. All crashes handled.
		WATCH: Postman is the industry standard for testing APIs. Learn it well — you will use it in every job.

WEEK 2 CHECKPOINT

- Why two tables instead of one? What breaks if you merge them?
- Walk through step by step what happens when a corrupted PDF hits /upload.
- What is uvicorn actually doing that FastAPI alone cannot do?
- Why is separation of concerns (rag_pipeline.py vs api.py) important?

WEEK 2 LINKEDIN POST

Turned last week's RAG script into an actual backend. FastAPI endpoints for PDF upload and document query, Postgres for document metadata via Supabase. Spent more time on error handling than the actual feature — the unglamorous work that makes something usable instead of a demo.

WEEK 3 Streamlit Frontend + Multi-Agent System

Give it a real interface users can see and use. Then upgrade from one AI call to multiple specialized agents working together.

WHAT YOU WILL HAVE BY END OF WEEK 3

A working website where you can upload PDFs and chat with them. Under the hood: three agents — Retrieval Agent (finds relevant chunks), Verification Agent (checks if answer is actually in the chunks), Citation Agent (formats the final answer with sources). This is what separates your project from a basic chatbot.

DAY 1-2	GOAL: Build Streamlit frontend 1. pip install streamlit 2. Create frontend.py: import streamlit as st import requests st.title('Document Q&A; System') st.subheader('Upload a PDF') uploaded_file = st.file_uploader('Choose a PDF', type='pdf') if uploaded_file: response = requests.post('http://localhost:8000/upload', files={'file': uploaded_file}) st.success(f'Uploaded: {uploaded_file.name}') st.subheader('Ask a Question') question = st.text_input('Your question') if st.button('Ask') and question: response = requests.post(f'http://localhost:8000/query?question={question}') st.write(response.json()['answer']) 3. Run: streamlit run frontend.py 4. You now have a real website at http://localhost:8501	TOOLS: streamlit BUILD: Website running. Upload PDF. Ask question. See answer. It looks and works like a real product. WATCH: You need BOTH running: uvicorn for the backend AND streamlit for the frontend — in two separate terminal windows.
DAY 3-4	GOAL: Build the multi-agent system 1. pip install langchain langchain-google-genai 2. Create agents.py with three functions: RETRIEVAL AGENT: takes question + collection → returns top 5 chunks (more than before) VERIFICATION AGENT: takes chunks + question → asks Gemini 'does this context contain enough info to answer?' → returns yes/no + reason CITATION AGENT: takes answer + chunks → formats final response with source references 3. Chain them together: def answer_question(question, collection): chunks = retrieval_agent(question, collection) verified = verification_agent(chunks, question) if not verified['sufficient']: return 'This document does not contain enough information to answer this question.' return citation_agent(verified['answer'], chunks) 4. Replace your /query endpoint to use answer_question() instead	TOOLS: LangChain BUILD: Three agents working. Verification agent correctly says 'insufficient' when question is outside the document. WATCH: This is what makes your project an 8.5/10 — the verification agent prevents hallucination, which is a real enterprise problem.

DAY 5	GOAL: Add conversation history	TOOLS: Streamlit session state
	Right now every question is independent. Add memory: if 'messages' not in st.session_state: <pre>st.session_state.messages = []</pre> for message in st.session_state.messages: with st.chat_message(message['role']): <pre>st.write(message['content'])</pre> if question := st.chat_input('Ask about your document'): <pre>st.session_state.messages.append({'role': 'user', 'content': question})</pre> answer = # call your API <pre>st.session_state.messages.append({'role': 'assistant', 'content': answer})</pre> Now it looks and behaves like a real chat interface.	BUILD: Chat interface working. Previous messages visible. Looks like ChatGPT but for your own documents.
DAY 6-7	GOAL: Polish and stress test	TOOLS: All tools
	1. Upload a 100+ page PDF — what happens? Does it timeout? Fix it. 2. Upload a scanned PDF with only images — text extraction returns nothing. Handle this case gracefully. 3. Ask the same question 10 times — are answers consistent? 4. Test in Chrome AND Firefox 5. Show it to one other person and watch them use it — where do they get confused? 6. Fix every issue you find 7. Add a loading spinner in Streamlit while waiting for AI response: with st.spinner('Thinking...'): answer = call_your_api()	BUILD: App stable under all test cases. Loading states working. Edge cases handled.
		WATCH: session_state persists data between Streamlit reruns — without it, everything resets every time the user clicks.
		WATCH: Watching someone else use your product is the single most valuable test. They will break things you never expected.

WEEK 3 CHECKPOINT

- What does the Verification Agent prevent that a simple one-call RAG cannot?
- Why use three agents instead of one big prompt?
- What happens to your Chroma data when the server restarts — and how did you fix it?

WEEK 3 LINKEDIN POST

Added a Streamlit frontend and upgraded to a multi-agent architecture. Three agents now: retrieval, verification, and citation. The verification agent is the interesting one — it refuses to answer if the document does not contain enough info, which prevents hallucination. That one feature took a day to get right.

WEEK 4 Docker + Deployment — Make It Live on the Internet

This week you go from 'runs on my laptop' to 'anyone in the world can use it'. This is what separates portfolio projects from toy projects.

WHAT YOU WILL HAVE BY END OF WEEK 4

A live public URL that anyone can open. Your full Multi-Agent RAG system running on the internet. Clean GitHub README with architecture diagram description. 2-minute demo video. LinkedIn capstone post. This is your portfolio centrepiece for the next 2 years.

DAY 1-2	GOAL: Dockerize your application	TOOLS: Docker Desktop
	1. Install Docker Desktop from docker.com — free 2. Create Dockerfile in rag-project root: FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt COPY . . CMD ['uvicorn', 'api:app', '--host', '0.0.0.0', '--port', '8000'] 3. Generate requirements.txt: pip freeze > requirements.txt 4. Build and test locally: docker build -t rag-app . docker run -p 8000:8000 rag-app 5. Your app should work exactly the same through Docker as it did directly	BUILD: App running inside Docker container locally. Same behaviour as before.
DAY 3-4	GOAL: Deploy to Railway	TOOLS: Railway
	1. Push all code to GitHub (make sure .env is in .gitignore) 2. Go to railway.app → sign up free → New Project → Deploy from GitHub repo 3. Select your rag-document-qa repo 4. Add environment variables in Railway dashboard (not in code): GEMINI_API_KEY=your_key DATABASE_URL=your_supabase_url 5. Railway automatically builds your Docker container and deploys it 6. You get a public URL like rag-document-qa.railway.app 7. Test it — upload a PDF from your phone. If it works, you have a live product.	BUILD: Live public URL. Works from your phone. Works from any browser. Show it to someone.
DAY 5	GOAL: Stress test the live deployment	TOOLS: Browser + phone
	Test everything on the live URL: 1. Upload a 50MB PDF — does it timeout or handle gracefully? 2. Upload from your phone — does the mobile interface work? 3. Ask 20 questions in a row — does it stay responsive? 4. Open two browser tabs simultaneously and use both — any conflicts? 5. Ask a question about something NOT in the document — does it correctly say insufficient info? 6. Write down every failure with: what broke, why it broke, how you fixed it These failures are your interview answers. 'The hardest problem I solved was...'	BUILD: Documented list of failures and fixes. This is more valuable than the build itself.
		WATCH: Never commit .env to GitHub. Set environment variables only in Railway dashboard. This is industry standard practice.
		WATCH: Every failure you document and fix is a real engineering story. Interviewers want this.

DAY 6	GOAL: Write README and record demo	TOOLS: GitHub, screen recorder
	README.md must include: 1. What the project does (2 sentences) 2. Architecture diagram (draw as ASCII or describe clearly) 3. Tech stack with reasons — WHY each tool, not just what 4. How to run locally (exact commands) 5. Live demo URL 6. What you learned / hardest problem solved Demo video (2 minutes max): 1. Show uploading a real PDF 2. Ask 3 questions — one the doc answers well, one it partially answers, one it correctly refuses 3. Show the verification agent refusing the out-of-scope question Record with OBS (free) or any screen recorder. Upload to YouTube unlisted. Link in README.	BUILD: README done. Demo video uploaded. GitHub repo clean and professional.
		WATCH: The README is what a recruiter reads before looking at code. Make it clear and specific.
DAY 7	GOAL: LinkedIn capstone post + portfolio update	TOOLS: LinkedIn
	Update LinkedIn: 1. Add project to Projects section with live URL and GitHub link 2. Update headline if needed 3. Write capstone post (template below) GitHub cleanup: 1. Make sure all code is committed 2. Add a proper .gitignore 3. Delete any test files, credentials, or junk 4. Pin this repo on your GitHub profile	BUILD: Project live on LinkedIn. GitHub pinned. Portfolio updated.
		WATCH: The live URL is everything. 'I built a RAG system' is a claim. A URL is proof.

WEEK 4 CHECKPOINT — Before you post publicly, answer these

- Why do containers use service names instead of 'localhost' to talk to each other?
- What were your 3 worst failures during stress testing — what was the ROOT cause?
- If someone asks 'what is the hardest technical problem you solved here' — do you have a specific real answer?
- Can you explain the difference between RAG and just asking ChatGPT the same question?

CAPSTONE LINKEDIN POST

Shipped a Multi-Agent Document Intelligence System — built and deployed over the last 4 weeks. Full RAG pipeline written from scratch in Python, wrapped in FastAPI with Postgres for document storage, multi-agent architecture with retrieval, verification, and citation agents, Streamlit frontend, containerized with Docker and deployed live on Railway. The verification agent was the most interesting part — it prevents the AI from hallucinating answers when the document does not contain enough information. That one feature took a day to debug and is what makes it production-grade. Most of the real learning happened fixing what broke during stress testing, not building the happy path. Live demo + repo in comments. #AI #RAG #Python #FastAPI #BuildingInPublic

COMPLETE INSTALL ORDER — Run These In This Exact Sequence

WEEK	COMMAND	WHAT IT INSTALLS
------	---------	------------------

1	<code>pip install pdfplumber</code>	PDF text extraction
1	<code>pip install google-generativeai python-dotenv</code>	Gemini API + .env file reading
1	<code>pip install chromadb</code>	Local vector database
2	<code>pip install fastapi uvicorn python-multipart</code>	Web backend + file upload support
2	<code>pip install psycpg2-binary sqlalchemy</code>	PostgreSQL connection
2	<code>pip install requests</code>	HTTP requests between services
3	<code>pip install streamlit</code>	Frontend website
3	<code>pip install langchain langchain-google-genai</code>	Agent orchestration framework
4	<code>pip freeze > requirements.txt</code>	Generate install list for Docker

FINAL PROJECT FOLDER STRUCTURE

rag-project/ ■■■ .env ← API keys (NEVER commit to GitHub) ■■■ .gitignore ← tells Git to ignore .env and venv/ ■■■ requirements.txt ← all pip installs (for Docker) ■■■ Dockerfile ← Docker build instructions ■■■ README.md ← your project documentation ■■■ main.py ← Week 1: standalone RAG script ■■■ rag_pipeline.py ← Week 2+: RAG functions (chunking, embedding, retrieval) ■■■ api.py ← Week 2+: FastAPI web backend ■■■ agents.py ← Week 3+: multi-agent system ■■■ frontend.py ← Week 3+: Streamlit website ■■■ chroma_db/ ← ChromaDB persistent storage folder ■■■ venv/ ← virtual environment (NEVER commit to GitHub)